

Architecture Blueprint & Feasibility Documentation: Hydra Mobile SDK for Android & iOS

The Hydra Mobile SDK enables **native mobile applications** (Android, iOS, Flutter) to interact with **Hydra Heads** for real-time, low-fee Layer-2 payments on Cardano. This document validates feasibility, defines the client workflow, specifies APIs and integration points, documents security and scalability strategies, and provides a complete system architecture blueprint.

Client Workflow (End-to-End)

Normal Operation Flow

Connect → Open/Join Head → Commit → In-Head Payments → Close → Fan-out

1. Connect

- Mobile SDK establishes a secure WebSocket (TLS) connection to `hydra-node`.
- Connection state and last known snapshot are restored if resuming.

2. Open / Join Head

- Client issues `openHead()` or `joinHead()` via HTTP API.
- SDK subscribes to Head lifecycle events.

3. Commit

- User commits UTxOs from L1 into the Head.
- SDK tracks commit confirmation via snapshot events.

4. In-Head Payments

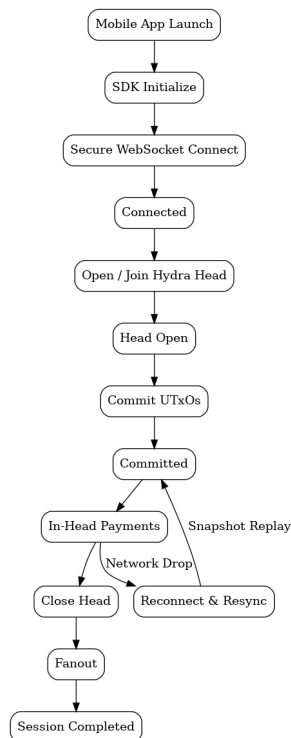
- L2 transactions created and signed locally.
- Transactions submitted via WebSocket.
- Validated via `TxValid` / `TxInvalid` events.

5. Close Head

- Client initiates Head closure.
- SDK monitors contestation and final snapshot.

6. Fan-out

- Final balances settled on L1.
- SDK confirms completion and clears local session state.



Failure & Recovery Paths

- **App Backgrounding / Kill** → Snapshot-based resume
- **Network Switch (WiFi ↔ Mobile)** → Reconnect + resync
- **Missed Events** → Snapshot replay

- **Partial Signing Failure** → Retry or user prompt

These flows ensure **crash safety and session continuity** under mobile constraints.

API & Integration Points

3.1 Hydra-Node Interfaces

WebSocket (Real-time):

- TxValid
- TxInvalid
- Snapshot
- HeadOpened
- HeadClosed

HTTP (Commands):

- /open
 - /join
 - /commit
 - /close
 - /fanout
 - /status
-

Wallet Bridge Options

To support signing on mobile:

- **Deep Links** – external wallet i
- **WebView Injection** – embedded wallet context

Supports:

- Hydra L2 signing keys

- Cardano L1 signing via compatible wallets
-

Transaction Build Adapters

- **Dart (Flutter)** tx builder
 - Deterministic transaction encoding
 - Abstracted signing interface
 - Pluggable signer backends
-

Dart Integration Mock

WebSocket Connection to Hydra Node

```
final channel = WebSocketChannel.connect(
  Uri.parse('wss://hydra-node.example.com'),
);

// Listen to Hydra events
channel.stream.listen((message) {
  final event = jsonDecode(message);

  switch (event['tag']) {
    case 'Snapshot':
      // update local head state
      break;

    case 'TxValid':
      // confirm transaction success
      break;

    case 'TxInvalid':
      // surface error to user
      break;
  }
}
```

```
});
```

HTTP Commit Call (Compatibility Example)

```
final response = await http.post(
  Uri.parse('https://hydra-node.example.com/commit'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({
    'utxo': [
      {'txId': '9f2c...ab', 'index': 0}
    ]
  }),
);

if (response.statusCode == 200) {
  // wait for Snapshot event over WebSocket
}
```

In-Head Transaction Submission

```
void submitInHeadTx(String signedTxCbor) {
  channel.sink.add(jsonEncode({
    'tag': 'NewTx',
    'transaction': signedTxCbor,
  }));
}
```

Hydra response (over WebSocket):

```
{"tag":"TxValid","transactionId":"a13f..."}
```

or

```
{"tag":"TxInvalid","reason":"ValidationError"}
```

Security & Scalability

Security Controls

- TLS with certificate pinning
- Hardware-backed key storage (Secure Enclave / Keystore)
- No private key material leaves device
- Strict message schema validation
- Replay protection at SDK and protocol layers

Scalability & Back-Pressure

- Buffered event pipeline
- Ordered message processing
- Back-pressure handling for snapshot floods
- Non-blocking WebSocket design

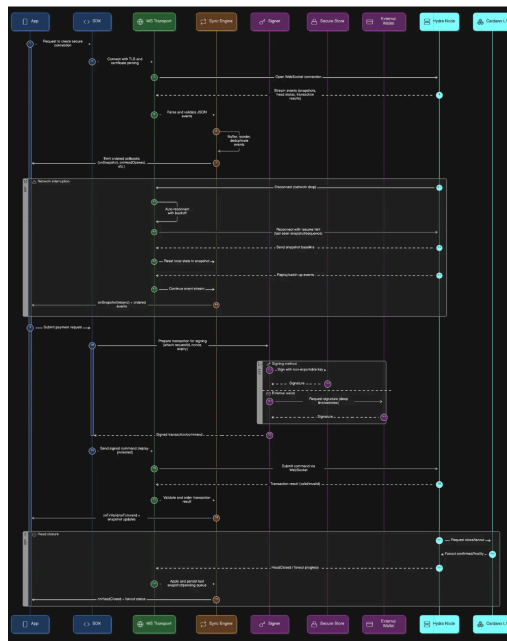
Hydra throughput is preserved by ensuring the SDK never becomes the bottleneck.

Risk Evaluation & Mitigation

Risk	Impact	Mitigation
App backgrounding	Missed events	Snapshot-based recovery
Network instability	Disconnections	Auto-reconnect & resync

Risk	Impact	Mitigation
Key compromise	Loss of funds	Hardware-backed storage
Hydra online requirement	Session drops	Provider-managed Heads
Event ordering issues	Invalid state	Sequenced processing

System Architecture Blueprint



Module Layout

- **Transport Layer** – WebSocket + HTTP clients
- **Sync Engine** – Snapshot recovery & ordering
- **Signer** – Hydra + L1 signing abstractions
- **Storage** – Persistent state & keys
- **Public SDK API** – Developer-facing interface

Components

The diagram explicitly defines the **sample application architecture** through the following components:

- **App**

The host mobile application invoking the SDK.

- **SDK**

Public-facing SDK API layer responsible for orchestration and lifecycle control.

- **WS Transport**

Manages TLS-secured WebSocket connectivity to the Hydra node, including certificate pinning and reconnect logic.

- **Sync Engine**

Ensures ordered event processing, snapshot buffering, deduplication, and deterministic state recovery.

- **Signer**

Abstract signing interface supporting:

- Hardware-backed local keys
- External wallet signing via deep link or WebView

- **Secure Store**

Persistent storage for snapshots, session state, and non-exportable key references.

- **External Wallet (Optional)**

Used when signing is delegated outside the SDK.

- **Hydra Node**

Unmodified hydra-node instance providing HTTP and WebSocket interfaces.

- **Cardano L1**

Final settlement layer during head closure and fan-out.

This layout constitutes a **complete sample mobile application architecture**, showing how the SDK is embedded and interacts with both Hydra and wallet infrastructure.

Normal Operation Flow

Connection & Initialization

- The App requests a secure connection.
- The SDK establishes a **TLS WebSocket connection with certificate pinning**.
- The Hydra node begins streaming events (snapshots, head status, transaction results).

Event Processing

- Incoming events are:
 - Parsed and schema-validated
 - Buffered, reordered, and deduplicated by the Sync Engine
- Ordered callbacks (e.g., `onSnapshot`, `onHeadOpened`) are emitted to the App.

This guarantees **deterministic state progression**, even under mobile constraints.

Explicit Error-Handling Flows

The diagram explicitly models **error-handling and recovery**, including:

Network Interruption

- WebSocket disconnect is detected by WS Transport.
- Auto-reconnect is attempted with backoff.
- SDK reconnects using a **resume hint** (last known snapshot sequence).
- Hydra node responds with a snapshot baseline.
- Sync Engine replays missed events.
- Local state is restored before resuming the event stream.

This ensures **crash-safe, interruption-tolerant recovery**, which is critical for mobile environments.

Event Loss / Ordering Issues

- Sync Engine buffers and reorders events.
 - Duplicate or out-of-order messages are discarded.
 - Application only receives **ordered, consistent state transitions**.
-

Transaction Failure

- `TxInvalid` events are propagated back to the SDK.
 - SDK validates ordering and surfaces failure to the App.
 - User can retry without state corruption.
-

Transaction Submission & Signing Flow

The diagram shows a **complete transaction lifecycle**:

1. App submits a payment request.
2. SDK prepares the transaction (nonce, expiry, replay protection).
3. Signing occurs via:
 - Hardware-backed local key, **or**
 - External wallet (deep link / WebView)
4. Signed transaction is returned to the SDK.
5. SDK submits the command via WebSocket.
6. Hydra node emits `TxValid` or `TxInvalid`.
7. SDK validates and updates local state accordingly.

This explicitly demonstrates **secure signing isolation** and **non-exportable key guarantees**.

Head Closure & L1 Settlement

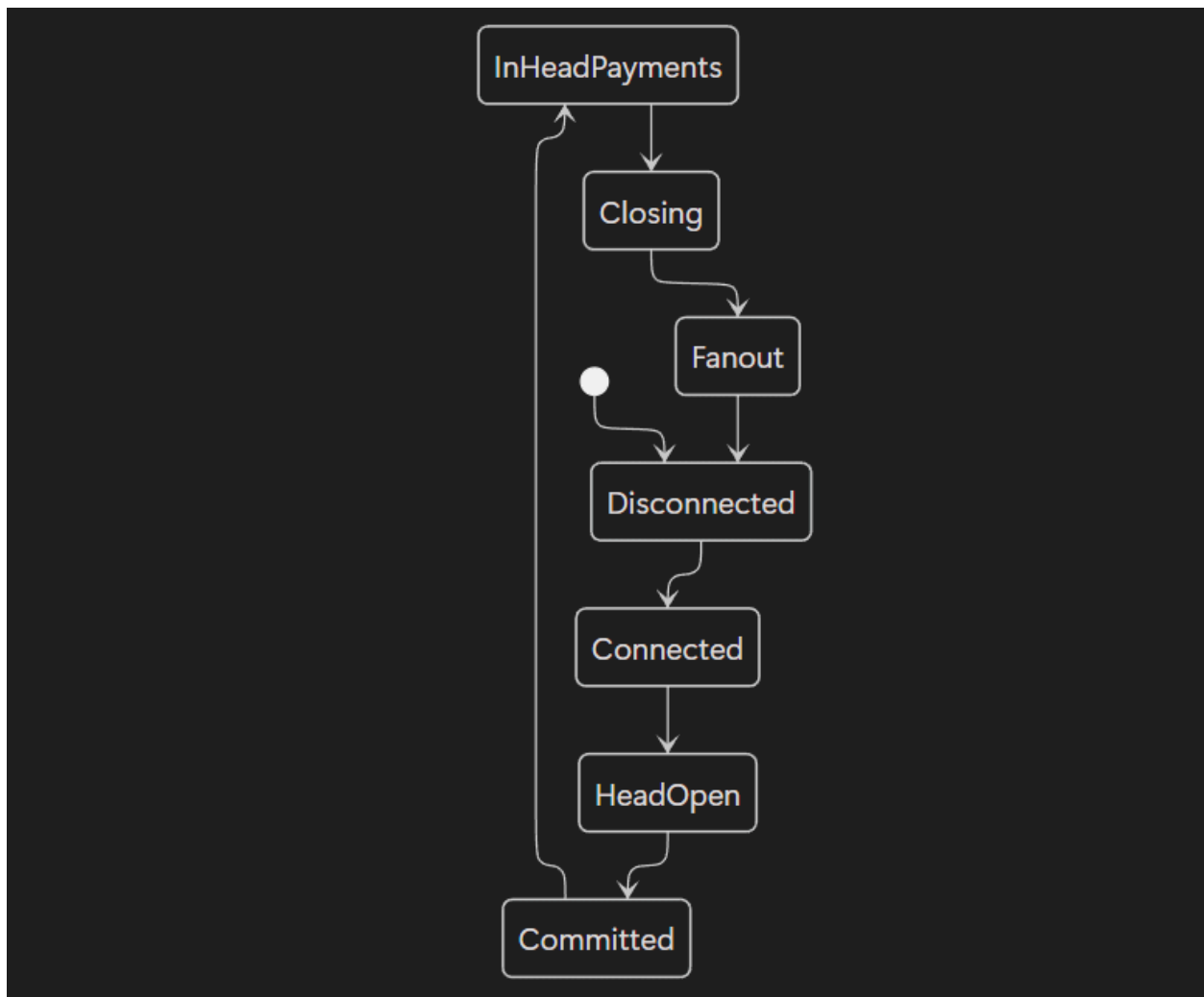
The diagram also captures the **head closure and fan-out flow**:

- App initiates close/fan-out.

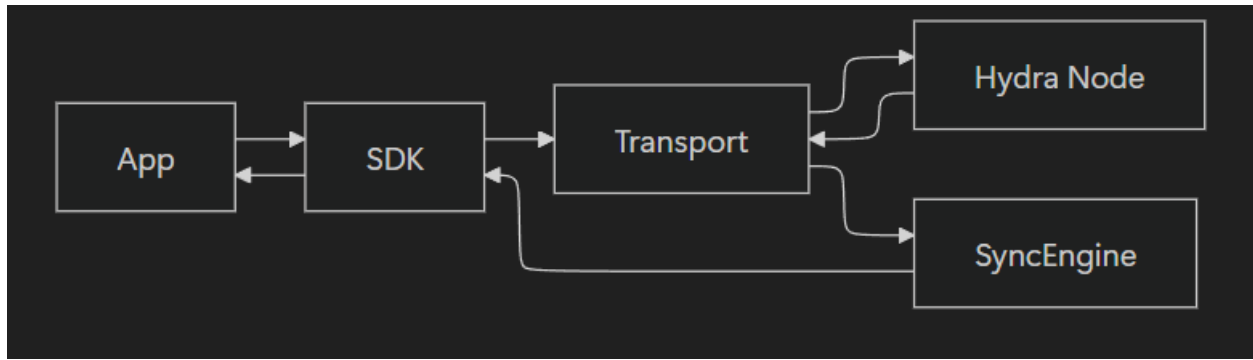
- Hydra node processes closure and finalizes balances.
- Cardano L1 confirms settlement.
- SDK persists final snapshot and clears pending queues.
- App receives `onHeadClosed` / fan-out confirmation.

This completes the **full L2 → L1 lifecycle**.

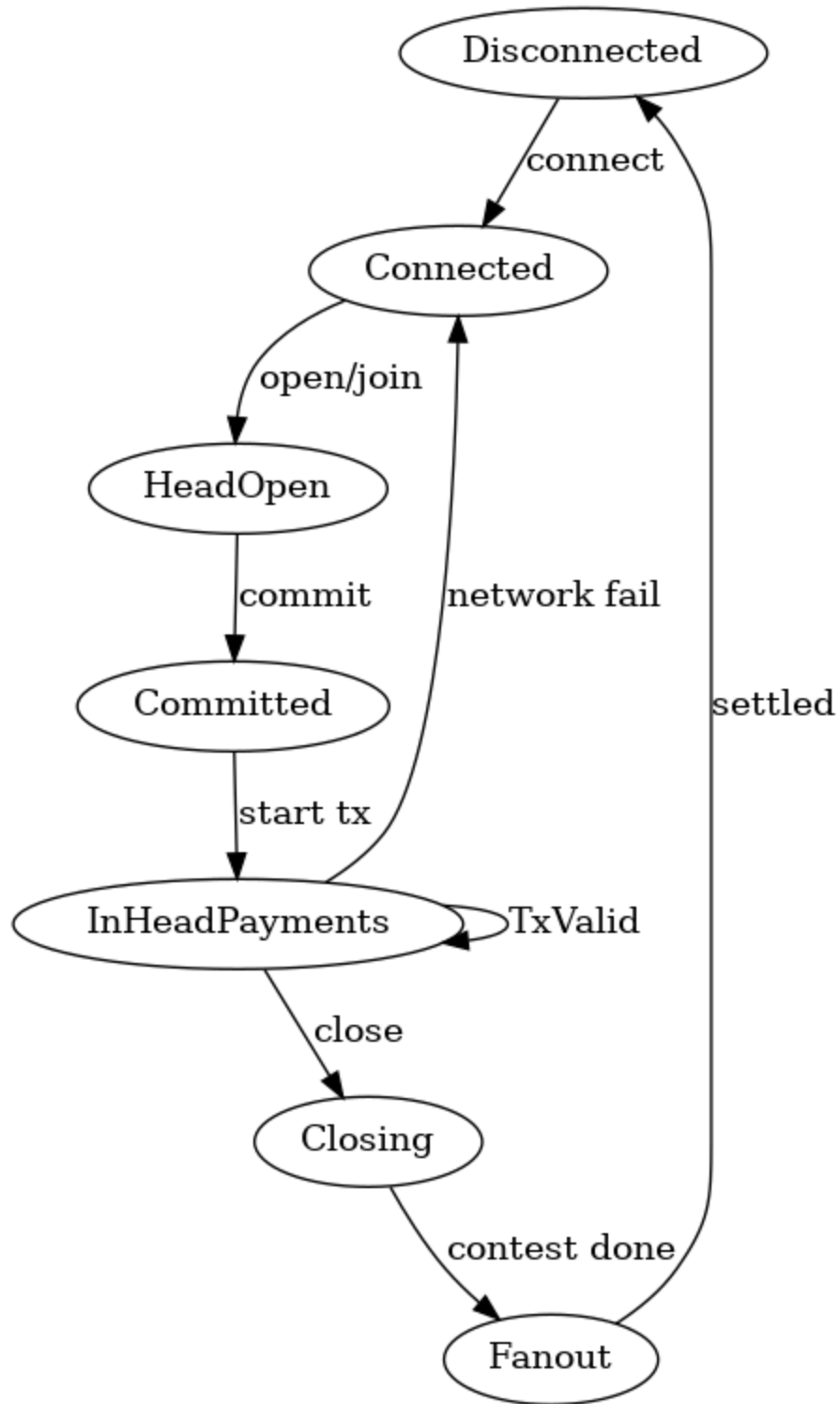
State Machine



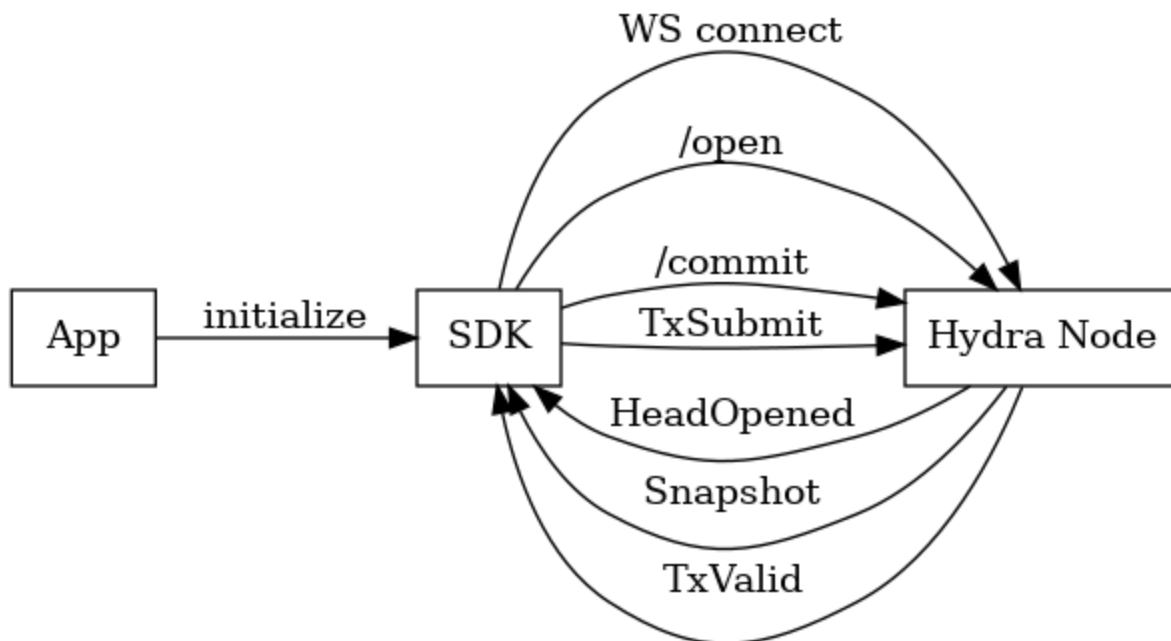
Message Pipeline



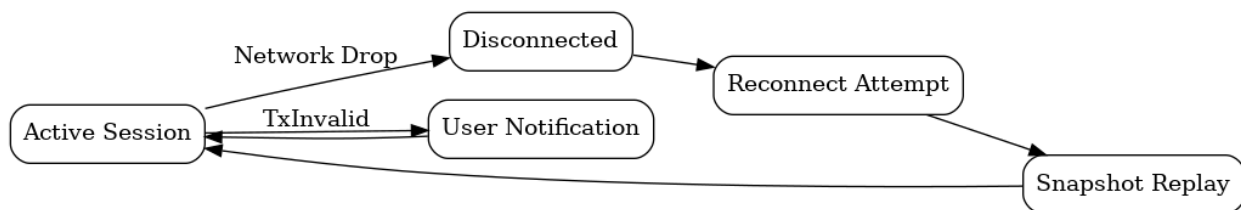
Client State Machine (Formal State Transitions)



Message Flow / SDK ↔ Hydra Integration



Error Handling & Recovery Flow



Sample Mobile Application Architecture

